

## Capítulo 3

# Construcción de la gramática producto de una gramática de concatenación de rango y un lenguaje regular

En el capítulo ?? se presentaron las gramáticas de concatenación de rango y se revisaron los antecedentes que abordan el SAT mediante formalismos de la teoría de lenguajes. En [5, 1] se asocia a cada fórmula booleana un autómata finito que reconoce sus interpretaciones verdaderas y una gramática que garantiza que las instancias de una misma variable tomen el mismo valor, y la satisfacibilidad se decide mediante el problema del vacío sobre la intersección de ambos lenguajes. Llevar esta estrategia a las gramáticas de concatenación de rango exige construir una gramática de concatenación de rango que reconozca la intersección del lenguaje de otra con un lenguaje regular.

La literatura provee tales construcciones para las gramáticas libres del contexto [2] y las de concatenación de rango simple [3]. En las gramáticas de concatenación de rango, una misma variable puede aparecer en varias posiciones de una cláusula, lo que introduce una dificultad que las construcciones anteriores no contemplan. Este capítulo aporta una construcción que, a partir de una gramática de concatenación de rango  $G$  y un autómata finito  $A$ , produce una nueva gramática de concatenación de rango  $G'$ , la *gramática producto*<sup>1</sup>, cuyo lenguaje es exactamente  $L(G) \cap L(A)$ .

---

<sup>1</sup>El término se usa por analogía con el autómata producto, la construcción estándar para la intersección de dos lenguajes regulares [6].

### 3.1. El rango sobre un autómata

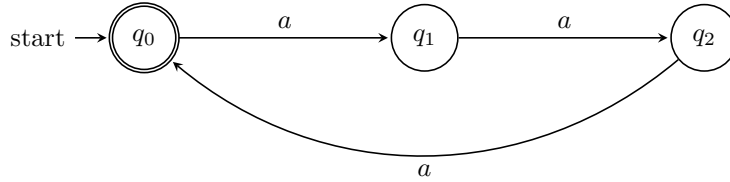
Antes de construir la gramática producto conviene precisar qué cadenas pertenecen a  $L(G) \cap L(A)$  y qué exige reconocerlas. La intersección contiene las cadenas que la gramática reconoce y el autómata acepta simultáneamente. Una cadena queda fuera en cuanto uno de los dos deja de reconocerla, de modo que decidir si pertenece a la intersección obliga a comprobar las dos condiciones sobre la misma cadena.

Comprobar cada condición por separado es directo. Sea la gramática  $G_0 = (N, T, V, P, Par)$ , donde  $N = \{Par\}$ ,  $T = \{a\}$ ,  $V = \{X\}$  y las cláusulas de  $P$  son

$$Par(aaX) \rightarrow Par(X) \quad \text{y} \quad Par(\varepsilon) \rightarrow \varepsilon.$$

$G_0$  genera el lenguaje  $\{a^{2n} \mid n \geq 0\}$ , esto es, las cadenas formadas por  $a$  de longitud par.

El autómata  $A_0$  sobre el alfabeto  $\{a\}$  tiene estados  $\{q_0, q_1, q_2\}$ , transiciones  $\delta(q_0, a) = q_1$ ,  $\delta(q_1, a) = q_2$  y  $\delta(q_2, a) = q_0$ , estado inicial  $q_0$  y estado final  $q_0$ , y reconoce las cadenas cuya longitud es múltiplo de tres.



Ante la cadena  $aaaaaa$ , el autómata la acepta porque, al leer sus símbolos, termina en  $q_0$ , y la gramática la reconoce porque, al aplicar sus cláusulas, la derivación termina en la cadena vacía.

La dificultad surge al exigir las dos condiciones a la vez sobre la misma cadena. El reconocimiento en el autómata avanza estado por estado; el de la gramática divide la cadena en rangos. Para coordinarlos haría falta saber, de cada rango, en qué estado está el autómata al empezar y al terminar de leer la subcadena que el rango delimita. El rango  $(i, j)$  no contiene esa información: marca posiciones dentro de una cadena fija, y no registra los estados del autómata.

Hace falta entonces una noción de rango que cargue la información del autómata. Para ello, cada subcadena se caracteriza por el par de estados entre los que el autómata transita al leerla. Para fijar ese par hace falta saber en qué estado termina el autómata al comenzar en un estado y leer una cadena, lo que da lugar a la siguiente definición.

**Definición 3.1.** Sea  $A = (Q, \Sigma, \delta, q_0, F)$  un autómata finito determinista. La función de transición  $\delta$  se extiende a cadenas mediante la *función de transición extendida*  $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$ , que se define por

$$\hat{\delta}(q, \varepsilon) = q \quad \text{y} \quad \hat{\delta}(q, wa) = \delta(\hat{\delta}(q, w), a)$$

para  $w \in \Sigma^*$  y  $a \in \Sigma$  [6]. Así,  $\hat{\delta}(q, u)$  es el estado en el que el autómata termina al comenzar en  $q$  y leer  $u$ .

Por ejemplo, en el autómata  $A_0$  se tiene

$$\hat{\delta}(q_0, a) = q_1, \quad \hat{\delta}(q_0, aa) = q_2, \quad \hat{\delta}(q_0, aaa) = q_0 \quad \text{y} \quad \hat{\delta}(q_1, aa) = q_0.$$

La función  $\hat{\delta}$  determina, para cada cadena y cada estado de partida, el estado en que el autómata termina, lo que permite definir el rango sobre el autómata como un par de estados y precisar cuándo una cadena lo realiza.

**Definición 3.2.** Un *rango sobre  $A$*  es un par de estados  $(q, q') \in Q \times Q$ . Una cadena  $u$  realiza el rango  $(q, q')$  si  $\hat{\delta}(q, u) = q'$ .

Un rango sobre  $A$  describe a la vez todas las cadenas que lo realizan. Cuando a una variable se le asocia el rango  $(q, q')$ , la gramática debe derivar la subcadena correspondiente, que a la vez debe realizar ese rango.

En el ejemplo anterior, cada  $a$  lleva al autómata  $A_0$  de  $q_0$  a  $q_1$ , de  $q_1$  a  $q_2$  y de  $q_2$  de vuelta a  $q_0$ . Así,  $a$  realiza  $(q_0, q_1)$ ,  $(q_1, q_2)$  y  $(q_2, q_0)$ ;  $aa$  realiza  $(q_0, q_2)$ ,  $(q_1, q_0)$  y  $(q_2, q_1)$ ; y  $aaa$  realiza  $(q_0, q_0)$ ,  $(q_1, q_1)$  y  $(q_2, q_2)$ .

En la sección siguiente, las nociones planteadas para las cadenas se generalizan al caso de los argumentos.

## 3.2. El rango de un argumento sobre un autómata

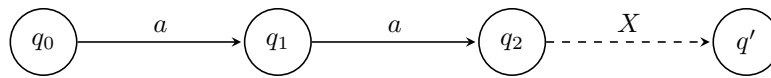
La sección anterior estableció cuándo una cadena realiza un rango sobre  $A$ . Un argumento de una cláusula es una secuencia de terminales y variables, y las variables no son símbolos que el autómata pueda leer. Hace falta entonces decir qué rango sobre  $A$  le corresponde a un argumento de esa forma.

**Definición 3.3.** Sea  $\alpha = \alpha_1 \cdots \alpha_n$  un argumento, donde cada  $\alpha_i \in T \cup V$ . Una *asignación de rangos sobre  $A$*  para  $\alpha$  es una función  $\rho$  que asigna a cada  $\alpha_i$  un rango sobre  $A$ ,  $\rho(\alpha_i) = (q_i, q'_i)$ , con dos condiciones:

1.  $q'_i = q_{i+1}$  para  $1 \leq i < n$ ;
2. si  $\alpha_i$  es un terminal  $a$ , entonces  $\delta(q_i, a) = q'_i$ .

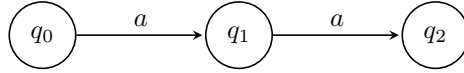
Cuando existe una asignación así, el argumento  $\alpha$  *realiza* el rango sobre  $A$   $(q_1, q'_n)$ .

Por ejemplo, sobre  $A_0$  se puede asignar al primer terminal  $a$  del argumento  $aaX$  el rango  $(q_0, q_1)$ , al segundo terminal  $a$  el rango  $(q_1, q_2)$ , y a la variable  $X$  un rango  $(q_2, q')$  con  $q' \in Q$ . La asignación se ve así:



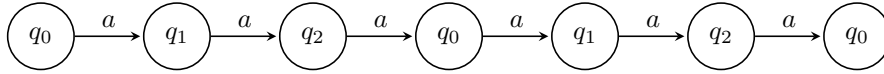
Las transiciones  $\delta(q_0, a) = q_1$  y  $\delta(q_1, a) = q_2$  satisfacen la condición 2 de la definición, que exige  $\delta(q_i, a) = q'_i$  para cada terminal  $a$ . El primer rango termina en  $q_1$ , donde empieza el segundo; el segundo termina en  $q_2$ , donde empieza el rango de  $X$ , satisfaciendo la condición 1. El argumento  $aaX$  realiza el rango  $(q_0, q')$ .

Las elecciones concretas para  $X$  revelan la interacción entre la gramática y el autómata. Si  $X = \varepsilon$ , entonces  $aaX = aa$  realiza  $(q_0, q_2)$ . La asignación se ve así:



$G_0$  deriva  $aa$  porque tiene longitud par, pero  $A_0$  no la acepta porque no termina en el estado final  $q_0$ . La cadena no pertenece a la intersección.

Si  $X = aaaa$ , entonces  $aaX = aaaaaa$  realiza  $(q_0, q_0)$ . La asignación se ve así:



$G_0$  deriva  $aaaaaa$  y  $A_0$  la acepta. La cadena pertenece a la intersección  $L(G_0) \cap L(A_0)$ .

Si  $X = aaa$ ,  $G_0$  no deriva  $aaa$  por su longitud impar, así que esa elección no aparece en ninguna derivación de  $G_0$ .

La sección siguiente presenta la construcción de la gramática producto sobre una subclase de gramáticas de concatenación de rango que reconoce los mismos lenguajes.

### 3.3. Construcción de la gramática producto

En esta sección se desarrolla la construcción de la gramática producto. Primero se presenta el algoritmo. A continuación se impone una restricción sobre la gramática de entrada. Después se establece una condición para que las ocurrencias de una misma variable reciban el mismo rango. Luego se da el pseudocódigo. Por último se demuestra su correctitud y se analiza su complejidad.

#### 3.3.1. Algoritmo

El objetivo es producir una gramática  $G' = (N', T, V, P', S')$  que reconozca las cadenas que  $G$  reconoce y  $A$  acepta. Una cadena  $w$  pertenece a  $L(A)$  si el rango sobre  $A$  que  $w$  realiza parte del estado inicial y termina en un estado final, así que  $G'$  no solo debe reconocer las cadenas de  $L(G)$ , sino reflejar también qué rango sobre  $A$  realiza cada argumento de cada predicado durante la derivación. Para reflejarlo, los predicados de  $G'$  se diferencian según el rango sobre  $A$  que realiza cada uno de sus argumentos: cada predicado de  $G$  da lugar a un predicado de  $G'$  por cada vector de rangos sobre  $A$ .

**Definición 3.4.** Sean  $B$  un predicado de  $G$  de aridad  $k$  y  $\vec{q} = ((q_1, q'_1), \dots, (q_k, q'_k))$  un vector de  $k$  rangos sobre  $A$ , uno por cada argumento de  $B$ . El par  $(B, \vec{q})$  se llama *predicado indexado* y se denota  $B[\vec{q}]$ .

Por ejemplo, como  $A_0$  tiene tres estados, la gramática  $G'$  de  $G_0$  sobre  $A_0$  tiene nueve predicados indexados  $Par[(q_i, q_j)]$ , uno por cada par  $(q_i, q_j)$  de estados de  $A_0$ :

$$\begin{array}{lll} Par[(q_0, q_0)], & Par[(q_0, q_1)], & Par[(q_0, q_2)], \\ Par[(q_1, q_0)], & Par[(q_1, q_1)], & Par[(q_1, q_2)], \\ Par[(q_2, q_0)], & Par[(q_2, q_1)], & Par[(q_2, q_2)]. \end{array}$$

El predicado inicial de  $G'$  es un predicado  $S'$  de aridad 1. El conjunto de predicados  $N'$  consta de los predicados indexados y del predicado inicial  $S'$ .

Las cláusulas de  $G'$  se generan a partir de las cláusulas de  $G$  y de las asignaciones de rangos sobre  $A$  a sus argumentos. Para cada cláusula

$$B_0(\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1(\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m(\alpha_{m,1}, \dots, \alpha_{m,k_m})$$

de  $G$  y cada asignación de rangos sobre  $A$  a cada argumento  $\alpha_{j,i}$ , se obtiene una cláusula de  $G'$  indexando cada predicado  $B_j$  por el vector  $\vec{q}_j$  cuya componente  $i$ -ésima es el rango  $(q_{j,i}, q'_{j,i})$  que realiza el argumento  $\alpha_{j,i}$  bajo su asignación:

$$B_0[\vec{q}_0](\alpha_{0,1}, \dots, \alpha_{0,k_0}) \rightarrow B_1[\vec{q}_1](\alpha_{1,1}, \dots, \alpha_{1,k_1}) \cdots B_m[\vec{q}_m](\alpha_{m,1}, \dots, \alpha_{m,k_m}).$$

En la misma gramática de ejemplo, si en el argumento  $aaX$  de la cláusula  $Par(aaX) \rightarrow Par(X)$  se asigna  $(q_0, q_1)$  al primer  $a$ ,  $(q_1, q_2)$  al segundo  $a$  y  $(q_2, q_0)$  a la variable  $X$  de la cabeza, el argumento  $aaX$  realiza el rango  $(q_0, q_0)$ . Si el argumento  $X$  del cuerpo realiza el rango  $(q_2, q_0)$ , la cláusula que se obtiene en  $G'$  es:

$$Par[(q_0, q_0)](aaX) \rightarrow Par[(q_2, q_0)](X).$$

La cláusula  $Par(aaX) \rightarrow Par(X)$  produce una cláusula en  $G'$  por cada combinación de una asignación al argumento  $aaX$  con una asignación al argumento  $X$  del cuerpo. El argumento  $aaX$  admite nueve asignaciones, una por cada rango  $(q_i, q_j)$  que realiza, y el argumento  $X$  del cuerpo admite otras nueve. Cada una de las nueve cabezas  $Par[(q_i, q_j)]$  se combina con cada uno de los nueve cuerpos  $Par[(q_k, q_l)]$ , lo que produce  $9 \times 9 = 81$  cláusulas:

$$\begin{array}{l} Par[(q_0, q_0)](aaX) \rightarrow \begin{cases} Par[(q_0, q_0)](X), \\ \vdots \\ Par[(q_2, q_2)](X), \end{cases} \\ \vdots \\ Par[(q_2, q_2)](aaX) \rightarrow \begin{cases} Par[(q_0, q_0)](X), \\ \vdots \\ Par[(q_2, q_2)](X). \end{cases} \end{array}$$

Cuando el cuerpo de una cláusula de  $G$  es vacío, solo se indexa la cabeza: la cláusula  $B(\alpha_1, \dots, \alpha_k) \rightarrow \varepsilon$  produce las cláusulas  $B[\vec{q}](\alpha_1, \dots, \alpha_k) \rightarrow \varepsilon$ , una por cada asignación de rangos a sus argumentos. El argumento  $\varepsilon$  realiza solo el rango  $(q, q)$  para cada estado  $q$ , así que la cláusula  $Par(\varepsilon) \rightarrow \varepsilon$  produce tres cláusulas en  $G'$ , una por cada estado del autómata:

$$\begin{aligned} Par[(q_0, q_0)](\varepsilon) &\rightarrow \varepsilon, \\ Par[(q_1, q_1)](\varepsilon) &\rightarrow \varepsilon, \\ Par[(q_2, q_2)](\varepsilon) &\rightarrow \varepsilon. \end{aligned}$$

El conjunto de cláusulas  $P'$  consta de las cláusulas anteriores y de las del predicado inicial  $S'$ , que se presentan a continuación.

El autómata  $A$  acepta una cadena cuando el rango que realiza parte del estado inicial  $q_0$  y termina en un estado final. Por eso la construcción solo usa los predicados indexados  $S[(q_0, q_F)]$  cuyo rango parte de  $q_0$  y termina en un estado final  $q_F \in F$ . Para cada  $q_F \in F$ ,  $G'$  tiene una cláusula

$$S'(X) \rightarrow S[(q_0, q_F)](X),$$

donde  $S$  es el predicado inicial de  $G$ . Una cadena  $w$  pertenece a  $L(G')$  si  $S'(w)$  se deriva a la cadena vacía.

En el ejemplo de  $G_0$  sobre  $A_0$ ,  $F = \{q_0\}$ , y el rango  $(q_0, q_0)$  parte del estado inicial y termina en un estado final. El predicado inicial  $S'$  tiene la cláusula:

$$S'(X) \rightarrow Par[(q_0, q_0)](X).$$

La construcción conserva todas las cláusulas que produce, tanto las que provienen de las cláusulas de  $G$  como las del predicado inicial  $S'$ . Las que participan en el reconocimiento de una cadena se determinan durante la derivación, no durante la construcción.

La construcción produce los predicados  $N'$ , las cláusulas  $P'$  y el predicado inicial  $S'$ , y mantiene los terminales  $T$  y las variables  $V$  de  $G$ . Los cinco componentes completan la tupla  $G' = (N', T, V, P', S')$ , con lo que la construcción de la gramática producto termina.

Para ilustrar el reconocimiento en la gramática producto  $G'$ , se describe paso a paso la derivación de la cadena  $aaaaaa \in L(G_0) \cap L(A_0)$ :

$$\begin{aligned} S'(aaaaaa) &\Rightarrow Par[(q_0, q_0)](aaaaaa) \\ &\Rightarrow Par[(q_2, q_0)](aaaa) \\ &\Rightarrow Par[(q_1, q_0)](aa) \\ &\Rightarrow Par[(q_0, q_0)](\varepsilon) \\ &\Rightarrow \varepsilon. \end{aligned}$$

La figura 3.1 representa el rango sobre  $A_0$  en cada paso. En el primer paso se aplica la cláusula del predicado inicial  $S'$ . Las cláusulas de los pasos 2, 3 y 4 provienen de  $Par(aaX) \rightarrow Par(X)$  con asignaciones distintas al argumento

$aaX$ , y la del paso 5 proviene de  $Par(\varepsilon) \rightarrow \varepsilon$ . Como la derivación de  $S'(aaaaaa)$  termina en la cadena vacía,  $G'$  reconoce  $aaaaaa$ .

La derivación de  $aaaaaa$  usa solo unas pocas de las cláusulas de  $G'$ ; las demás no participan en su reconocimiento. La construcción genera, además, muchos predicados y cláusulas que no aparecen en el reconocimiento de ninguna cadena de  $L(G_0) \cap L(A_0)$ . Todos los rangos de la derivación anterior terminan en el estado final  $q_0$ ; los predicados indexados cuyo rango termina en otro estado, como  $Par[(q_0, q_1)]$  o  $Par[(q_1, q_2)]$ , quedan sin uso, junto con las cláusulas que los tienen en la cabeza, como por ejemplo  $Par[(q_1, q_2)](aaX) \rightarrow Par[(q_0, q_2)](X)$ .

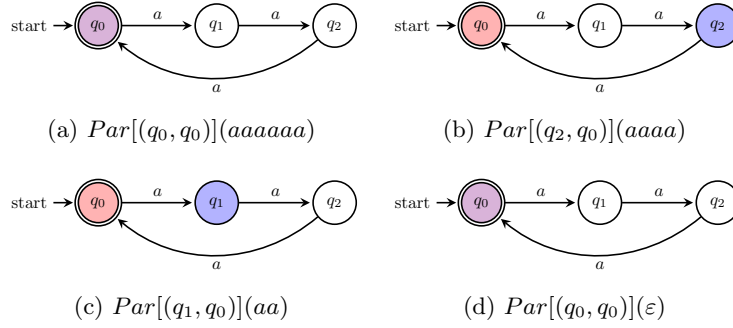


Figura 3.1: Rango sobre  $A_0$  en cada paso de la derivación de  $aaaaaa$ . El color azul corresponde al estado de partida del rango; el rojo, al de llegada; el violeta, a ambos cuando coinciden.

Por conveniencia, la construcción anterior se restringe a una clase de gramáticas de entrada que evita casos aparte. La sección siguiente la precisa.

### 3.3.2. Restricción sobre la gramática de entrada

Cada predicado se indexa por los rangos sobre  $A$  de sus argumentos. El cuerpo de una cláusula fija los rangos de las variables que contiene. Si una variable apareciera en la cabeza pero no en el cuerpo, el cuerpo no fijaría su rango, y habría que tratarla como un caso aparte.

Para evitarlo, las gramáticas de entrada se restringen a las que cumplen una propiedad sobre sus cláusulas: en cada cláusula, toda variable de la cabeza aparece también en el cuerpo.

**Definición 3.5.** Una gramática de concatenación de rango es *top-down non-erasing* si en cada cláusula toda variable de la cabeza aparece también en el cuerpo.

Por ejemplo, las cláusulas de  $G_0$  cumplen la propiedad:  $Par(aaX) \rightarrow Par(X)$  tiene la variable  $X$  en la cabeza y en el cuerpo, y  $Par(\varepsilon) \rightarrow \varepsilon$  no tiene variables. Una cláusula como  $A(XY) \rightarrow B(X)$  no la cumple, pues  $Y$  figura en la cabeza y falta en el cuerpo.

Exigir la propiedad no descarta ningún lenguaje: toda gramática de concatenación de rango admite una equivalente que la cumple [4, 7].

Además de la restricción sobre la gramática de entrada, el algoritmo necesita una condición sobre los rangos que asigna. La sección siguiente la precisa.

### 3.3.3. Variables repetidas en una cláusula

En las gramáticas de concatenación de rango, una misma variable puede aparecer varias veces en una cláusula. En la construcción de la sección 3.3.1, cada argumento se indexa por separado, sin que se fije el mismo rango para las ocurrencias de una misma variable, que representan una sola cadena. Sin una condición que lo imponga, se les pueden asignar rangos distintos, y entonces  $G'$  reconocería cadenas que no están en la intersección.

Por eso es necesario añadir la siguiente condición: en cada cláusula, a todas las ocurrencias de una variable se les asigna el mismo rango. Más formalmente, si una variable  $X$  aparece en dos argumentos de la cláusula con asignaciones de rangos  $\rho$  y  $\rho'$ , entonces  $\rho(X) = \rho'(X)$ .

Por ejemplo, si a  $G_0$  se le agrega un predicado *Cinco*, que reconoce las cadenas de longitud múltiplo de cinco, y una cláusula inicial que combina *Par* y *Cinco* sobre la misma variable, la gramática resultante es  $G_c = (N, T, V, P, S)$ , donde  $N = \{S, Par, Cinco\}$ ,  $T = \{a\}$ ,  $V = \{X\}$  y las cláusulas de  $P$  son

$$\begin{aligned} S(X) &\rightarrow Par(X) Cinco(X), \\ Par(aaX) &\rightarrow Par(X), \quad Par(\varepsilon) \rightarrow \varepsilon, \\ Cinco(aaaaaX) &\rightarrow Cinco(X), \quad Cinco(\varepsilon) \rightarrow \varepsilon. \end{aligned}$$

$G_c$  reconoce las cadenas que *Par* y *Cinco* reconocen a la vez, las de longitud múltiplo de diez.

Sobre  $A_0$ , la cadena  $a^{10}$  realiza el rango  $(q_0, q_1)$ , porque su longitud deja resto uno al dividir entre tres, y el rango no termina en el estado final. Su longitud es múltiplo de dos y de cinco, pero no de tres, así que no pertenece a la intersección.

Sin añadir la condición existe la cláusula

$$S[(q_0, q_0)](X) \rightarrow Par[(q_0, q_1)](X) Cinco[(q_0, q_1)](X),$$

con el predicado de la cabeza indexado en el rango  $(q_0, q_0)$  y los predicados del cuerpo indexados en el rango  $(q_0, q_1)$ . Como  $a^{10}$  realiza  $(q_0, q_1)$  y la reconocen tanto *Par* como *Cinco*, los dos cuerpos se derivan en la cadena vacía, y  $G'$  reconoce  $a^{10}$ , la cual está fuera de la intersección.

Con la condición, las ocurrencias de  $X$  llevan el mismo rango, y la única cláusula válida con la cabeza indexada en el rango  $(q_0, q_0)$  es

$$S[(q_0, q_0)](X) \rightarrow Par[(q_0, q_0)](X) Cinco[(q_0, q_0)](X).$$

Como  $a^{10}$  realiza  $(q_0, q_1)$  y no  $(q_0, q_0)$ ,  $Par[(q_0, q_0)](a^{10})$  no se deriva y  $a^{10}$  queda fuera del lenguaje de  $G'$ , como debería ser. La gramática producto reconoce



entonces exactamente las cadenas de longitud múltiplo de dos, tres y cinco, es decir de treinta.

La repetición de variables es justo lo que separa a las gramáticas de concatenación de rango de las simples, y la condición es lo que el algoritmo para las simples no necesita. Imponerla es lo que permite tratar gramáticas de concatenación de rango generales.

# Bibliografía

- [1] Manuel Aguilera López. Problema de la satisfacibilidad booleana de concatenación de rango simple. In *Jornada Científica Estudiantil MatCom 2016*, pages 1–5, La Habana, Cuba, 2016. Facultad de Matemática y Computación, Universidad de La Habana.
- [2] Yehoshua Bar-Hillel, Micha Perles, and Eli Shamir. On formal properties of simple phrase structure grammars. *Zeitschrift für Phonetik, Sprachwissenschaft und Kommunikationsforschung*, 14(2):143–172, 1961.
- [3] Eberhard Bertsch and Mark-Jan Nederhof. On the complexity of some extensions of RCG parsing. In *Proceedings of the Seventh International Workshop on Parsing Technologies (IWPT 2001)*, pages 66–77, Beijing, China, 2001.
- [4] Pierre Boullier. Proposal for a natural language processing syntactic backbone. Technical Report RR-3342, INRIA-Rocquencourt, France, 1998.
- [5] Alina Fernández Arias. El problema de la satisfacibilidad booleana libre del contexto. un algoritmo polinomial. Tesis de licenciatura, Facultad de Matemática y Computación, Universidad de La Habana, La Habana, Cuba, 2007.
- [6] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson Addison-Wesley, Boston, MA, 3 edition, 2006.
- [7] Laura Kallmeyer. *Parsing Beyond Context-Free Grammars*. Cognitive Technologies. Springer, Berlin, Heidelberg, 2010.